

Final Project: Fully automated ML project

Master's in Machine Learning 2025

Authors: Igor Vons, Endika Aguirre

Date: January 10th, 2025

Contents

1	Introduction	3
2	Dataset	3
2.1	Class Distribution	3
2.2	Features	4
2.2.1	Feature Distributions	4
2.2.2	Feature Correlations	5
3	Repository	7
3.1	Structure	7
3.1.1	Backend	8
3.1.2	Frontend	9
3.2	Remote references	9
4	Models	9
4.1	Model Selection	9
4.2	Validation & Optimization	9
4.2.1	Model Validation	10
4.2.2	Metrics	10
4.2.3	Hyperparameter Search Space	10
4.2.4	Experiments	10
4.3	Model Results	11
4.3.1	Performance Comparison	11
4.3.2	Best Model Configuration	12
4.3.3	Operational Metrics	12
4.4	Serialization	12
5	Automation Pipeline	12
5.1	Testing	12
5.2	Continuous Integration (CI)	13
5.3	Continuous Deployment (CD)	13
5.3.1	Backend Pipeline (API)	13
5.3.2	Frontend Pipeline (UI)	13
6	Monitoring	13
6.1	Prometheus	14
6.2	Grafana	14
7	Personal Server	15
7.1	Infrastructure Setup	15
7.2	Docker Compose Stack	16
7.3	Container Monitoring	16
7.4	Public Access via Cloudflare Tunnel	17
8	Repository Links	18

1 Introduction

This report details the end-to-end MLOps lifecycle for a binary classification system designed to distinguish between **Gonen** and **Jasmine** rice varieties. The project demonstrates a fully automated pipeline encompassing:

- **Data Engineering:** Handling tabular morphological data.
- **Model Development:** Training and serializing XGBoost/TabNet models using Optuna for hyperparameter tuning.
- **CI/CD Automation:** A Conditional Training pipeline using GitHub Actions.
- **Deployment:** Containerized serving via FastAPI and Docker on Hugging Face Spaces.
- **Observability:** Real-time monitoring with Prometheus and Grafana.

2 Dataset

The core task is supervised binary classification of an imbalanced dataset containing morphological measurements of rice grains.

2.1 Class Distribution

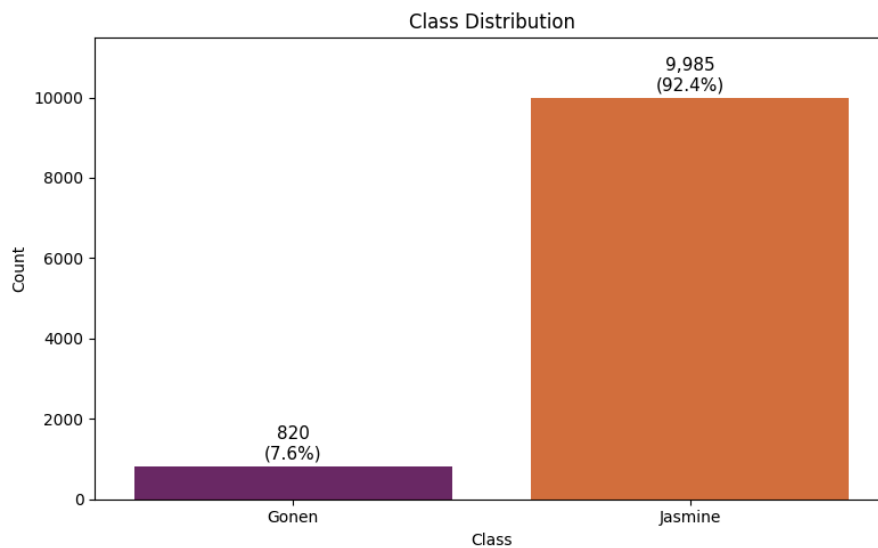


Figure 1: Class Distribution in the Final Dataset

The original dataset is mostly balanced, with 18185 samples total of which 8200 are **Gonen** (45.1%) and 9985 are **Jasmine** (54.9%). However, because this project aims to demonstrate handling of imbalanced datasets, we artificially increased the imbalance by downsampling the **Gonen** class to 820 samples, resulting in a final distribution of 820 **Gonen** (7.4%) and 9985 **Jasmine** (92.6%). A ratio of approximately 1:12.

This was performed in such a way that the data preserved its original distribution characteristics, ensuring that the model would still learn meaningful patterns despite the increased imbalance.

2.2 Features

- **Input:** The dataset features 10 measurements extracted from rice grain images. The author of the dataset didn't provide any information about what each feature represents, so we had to reverse engineer the meaning of each feature based on their names, common sense and comparing predictions with the registered values. The features are:
 - **Area:** Area of the rice grain. (mm^2)
 - **Perimeter:** Perimeter length of the rice grain. (mm)
 - **MajorAxisLength:** Length of the major axis of the ellipse that fits the rice grain. (mm)
 - **MinorAxisLength:** Length of the minor axis of the ellipse that fits the rice grain. (mm)
 - **ConvexArea:** Area of the convex hull that encloses the rice grain. (mm^2)
 - **EquivDiameter:** Diameter of a circle with the same area as the rice grain. (mm)
 - **Extent:** Ratio of the area of the rice grain to the area of its bounding box.
 - **Eccentricity:** Measure of how elongated the rice grain is.
 - **Roundness:** Measure of how round the rice grain is.
 - **AspectRatio:** Ratio of the major axis length to the minor axis length.
- **Output:** The target variable is the rice variety, encoded as 0 for **Gonen** and 1 for **Jasmine**.

2.2.1 Feature Distributions

Taking a look at the feature distributions by class, we can observe noticeable differences between **Gonen** and **Jasmine** rice grains. Features like **Roundness**, **MinorAxisLength**, and **AspectRatio** show clear correlation with the target class, indicating that, despite still having some partial overlap, the distributions are distinct enough to aid the model in learning effective decision boundaries.

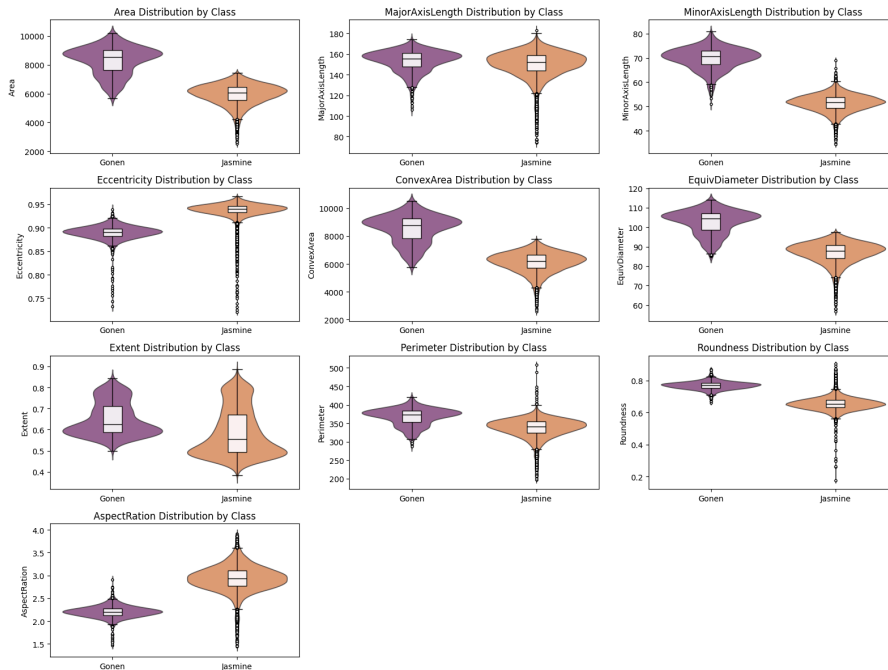


Figure 2: Feature Distributions by Class

Notable exceptions to this trend are `MajorAxisLength` and `Extent`, which exhibit a much higher degree of overlap between the two classes.

Another notable observation is that Jasmine features tend to have a greater amount of outliers, and they are generally more spread out, while Gonen features are more tightly clustered. We attribute this, partially, to the larger sample size of Jasmine, which in turn makes the quantiles stay closer to the mean, marking more points as outliers.

A further analysis via pairplot of 5 selected features (`Area`, `MajorAxisLength`, `MinorAxisLength`, `Eccentricity`, `ConvexArea`, selected to improve readability) reveals that while there is some overlap in the feature space, the two classes occupy two mostly separated clusters. This suggests that a well-trained model should be able to effectively separate the classes based on these features.

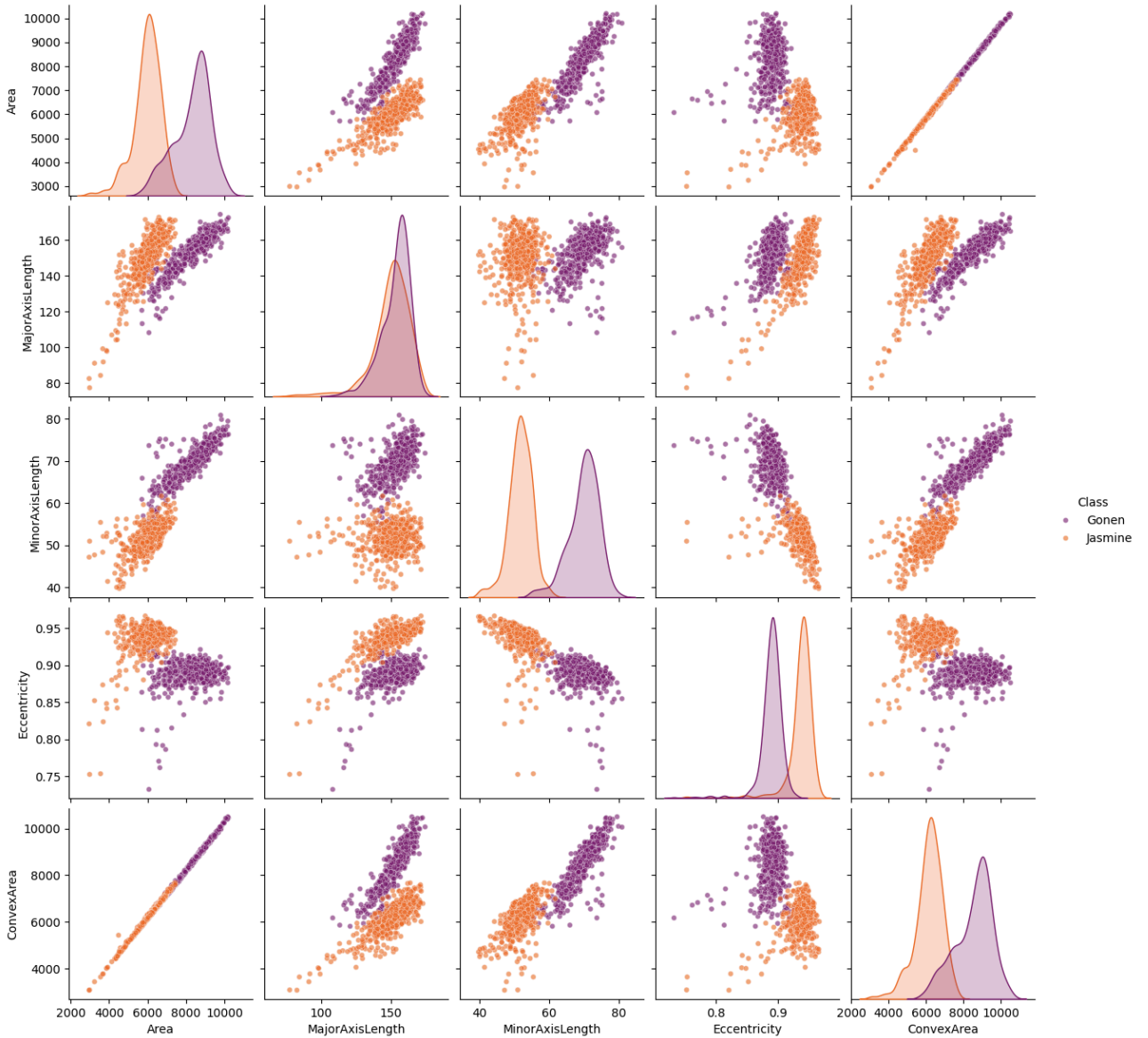


Figure 3: Feature Distributions by Class

2.2.2 Feature Correlations

A correlation matrix was computed to assess the relationships between features. We also added hierarchical clustering to group similar features together, and make the plot easier to interpret. This information

was also used during feature interpretability analysis, to understand which features could be used to compute the others.

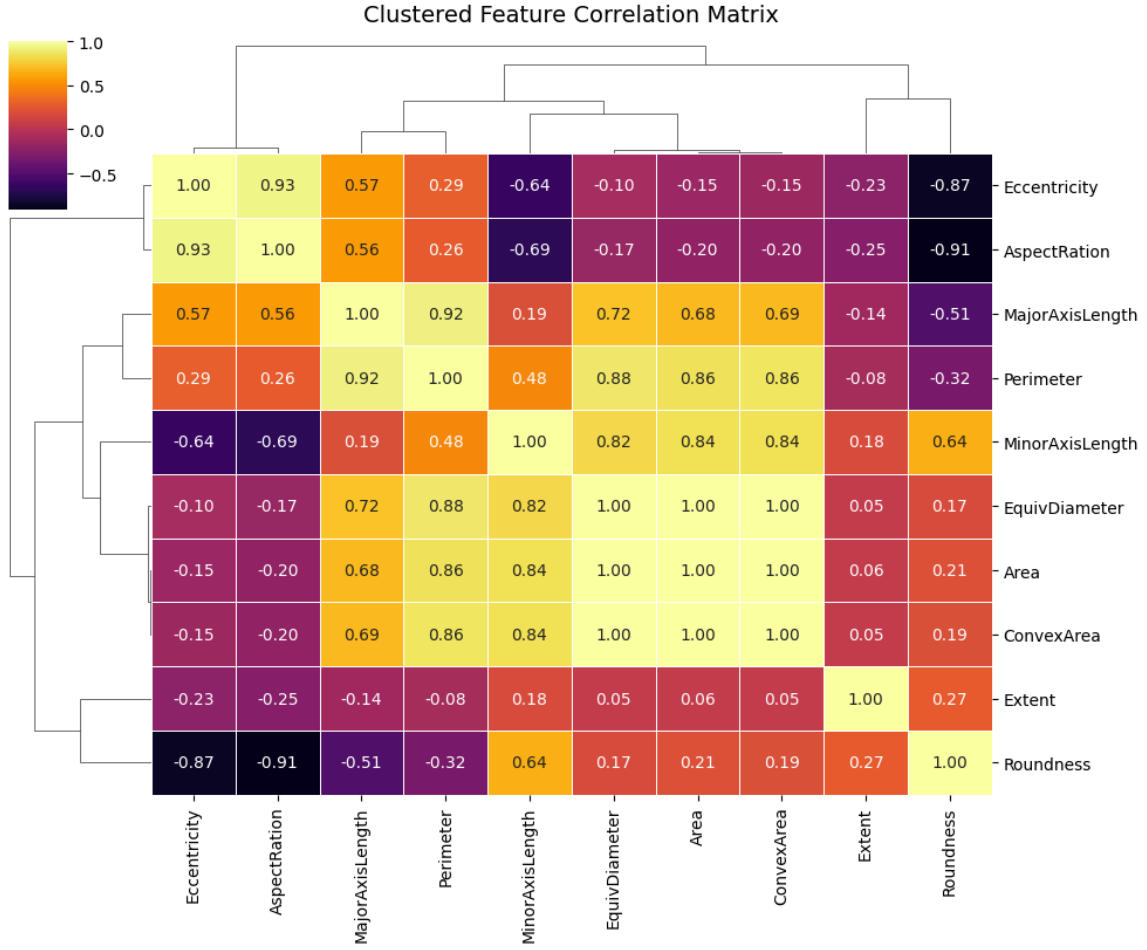


Figure 4: Feature Correlation Matrix with Hierarchical Clustering

Right away we observe very strong correlations: a 1 between Area, ConvexArea, and EquivDiameter; > 0.8 between those and Perimeter, MajorAxisLength and MinorAxisLength; > 0.9 for AspectRatio and Eccentricity; < -0.85 between Roundness and AspectRatio and Roundness and Eccentricity. And these are just the strongest ones. The most notable exception is Extent, which shows very weak correlations with all other features, indicating it captures a unique aspect of the data recollected.

With this information, and some reverse engineering, we established how each feature derives from the others.

- MajorAxisLength(L_M) and MinorAxisLength(L_m) are the axes of an ellipse that fits the grain shape. They can be treated as independent features.
- Area: $A = \pi \frac{L_M}{2} \frac{L_m}{2}$
- Perimeter: $P \approx \pi \left(\frac{L_M}{2} + \frac{L_m}{2} \right) \left(1 + \frac{3 \frac{(\frac{L_M}{2} - \frac{L_m}{2})^2}{(\frac{L_M}{2} + \frac{L_m}{2})^2}}{10 + \sqrt{4 - 3 \frac{(\frac{L_M}{2} - \frac{L_m}{2})^2}{(\frac{L_M}{2} + \frac{L_m}{2})^2}}} \right)$ *Ramanujan's II approximation*
- AspectRatio: $AR = \frac{L_M}{L_m}$

- Eccentricity: $e = \begin{cases} \sqrt{1 - \left(\frac{L_m}{L_M}\right)^2} & \text{if } AR \geq 1 \\ \sqrt{1 - \left(\frac{L_M}{L_m}\right)^2} & \text{if } AR < 1 \end{cases}$
- EquivDiameter: $ED = 2\sqrt{\frac{A}{\pi}}$
- Roundness: $R = 4\pi \frac{A}{P^2}$
- ConvexArea: $A_c = 1.02A$

We then were able to confirm these relationships by computing the derived features from the independent ones and comparing them to the original values, obtaining close matches in most cases.

The only feature that could not be derived from others was Extent. The interpretation we give it is that it measures how well the rice grain fills its bounding box, which is not directly related to the other geometric properties, but rather to the method of image acquisition, supposing the grain can be rotated or positioned differently within the bounding box.

$$A_{\text{bounding box}} = 2\sqrt{\frac{L_M}{2}\cos(\theta)^2 + \frac{L_m}{2}\sin(\theta)^2} \cdot 2\sqrt{\frac{L_M}{2}\sin(\theta)^2 + \frac{L_m}{2}\cos(\theta)^2}$$

$$\text{Extent} = \frac{A}{A_{\text{bounding box}}}$$

Figure 5: Extent Calculation Based on Bounding Box Area

These were later used in the final versions of the user interface, to provide an easier way of making requests by allowing users to input only the independent features, while the dependent ones were computed internally.

3 Repository

Before explaining the rest of the project (including automation, deployment, monitoring, and results), we provide an overview of the repository structure, so the reader can better follow any references mentioned.

3.1 Structure

The whole project is contained within a single GitHub repository, but organized into two separate branches with no shared history:

- **main**: Contains the source code for the application, including data processing, model training, API serving, and monitoring setup.
- **ui-frontend**: Contains a simple web-based user interface to interact with the deployed model API.
- **ui-frontend-react**: Contains an alternative version of the user interface built with Node.js, and also contains the docker compose to build and deploy the api + ui service in our private server.

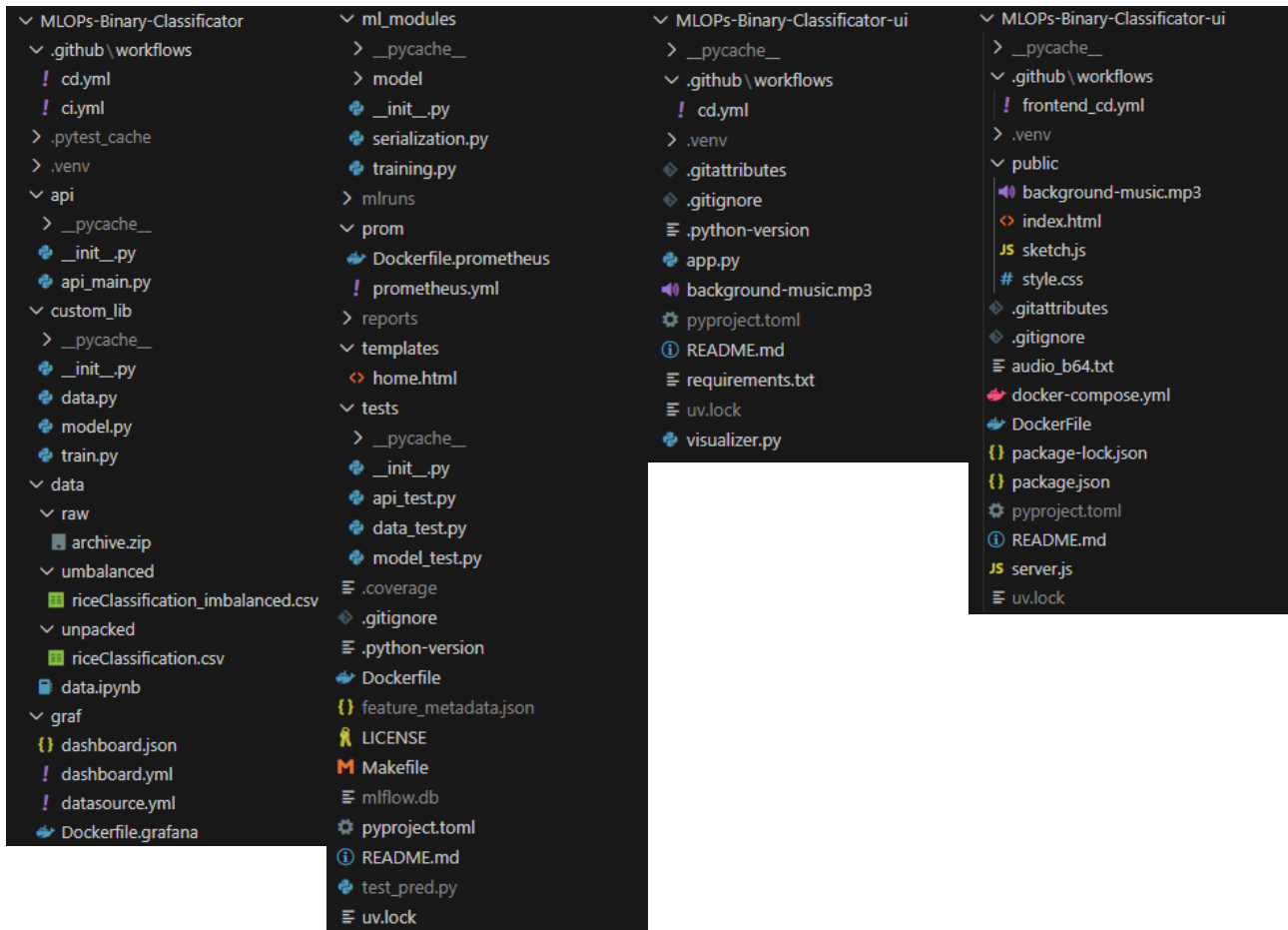


Figure 6: Repository Structure: Backend (left/center) and Frontend (center/right)

3.1.1 Backend

The backend code resides in the main branch, organized as follows:

- `github/workflows/`: GitHub Actions workflows for CI/CD and Conditional Training.
- `api/`: FastAPI application code for exposing the endpoint to serve model predictions.
- `custom_lib/`: The library containing the modules for calling model predictions from ONNX files, visualizing and treating data, and the training pipeline that later gets called from GitHub Actions.
- `data/`: Data processing and study notebook and dataset files.
- `graf/`: Grafana dashboard configuration files and docker setup for the monitoring interface deployment.
- `ml_modules/`: Model training, serialization, and evaluation logic. It also contains the serialized ONNX models inside a `models/` subfolder.
- `templates/`: HTML home template for the FastAPI application.
- `prom/`: Prometheus configuration files and docker setup for the monitoring middleware deployment.
- `tests/`: Unit and integration tests for data, model, and API components.
- `base` directory: Contains root-level files like `Dockerfile`, `uv` environment in `pyproject.toml` and `uv.lock`, `Makefile` with commands that are mainly used in the CI, `README.md` and `LICENSE`.

3.1.2 Frontend

The frontend code resides in the `ui-frontend` branch, organized as follows:

- `github/workflows/`: GitHub Actions workflow for CD of the frontend.
- `base` directory: Contains root-level files like the `app.py` gradio application that connects to the api endpoints, `visualization.py` which contains the `p5.js` code to visualize rice grains based on input features, and `requirements.txt` and `README.md` files required by the HuggingFace space where it will be deployed. There are also the `uv` environment files `pyproject.toml` and `uv.lock`, but they are excluded from the repo as they are not supported by HuggingFace Spaces. Instead, their contents are poured into `requirements.txt`.

3.2 Remote references

- **GitHub Repository**: Where the full source code is hosted, along with the CI/CD pipeline.
- **Docker Hub**: Where the Docker images for the API and monitoring stack are built and stored.
- **Render web services**: Where the docker containers are deployed to serve the API and monitoring dashboards.
- **Hugging Face Spaces**: Where the frontend user interface is deployed to interact with the API.
- **Private Server**: Where a parallel api and frontend deployment was hosted for the purpose of adding extra complexity to the project, and demonstrate deployment to custom infrastructure.

4 Models

Although in the dataset analysis we observed that the features had high dependencies between them, we still opted to use all features for training, as the deviations from ideal mathematical relationships could contain useful information for the model to learn from. In problems with a much larger number of features or where the dataset size is much larger, feature selection or dimensionality reduction techniques would be more appropriate.

4.1 Model Selection

In this project we developed and deployed both models discussed in the instructions (**XGBoost** and **TabNet**) to solve the binary classification task:

- **XGBoost (eXtreme Gradient Boosting)**: A gradient-boosted decision tree algorithm chosen for its efficiency and specialized optimization for tabular data. We use the `binary:logistic` objective function to minimize LogLoss. To counter the 1:12 class imbalance, we utilize the `scale_pos_weight` parameter. This modifies the loss function: $L = -(w_{\text{pos}} \cdot y \cdot \log(p) + (1 - y) \cdot \log(1 - p))$, heavily penalizing errors on the minority class (**Gonen**) during training.
- **TabNet**: A deep learning architecture designed to process tabular data by using sequential attention mechanisms to select features at each decision step, mimicking the interpretability of decision trees within a neural network framework.

4.2 Validation & Optimization

Hyperparameter tuning is orchestrated via **Optuna**, running 20 trials to maximize model performance. We structured our validation approach as follows:

4.2.1 Model Validation

We employ **Stratified K-Fold Cross-Validation (K=5)**. So that the training data is split into 5 folds, ensuring the 1:12 class imbalance is preserved in each split, ensuring the imbalanced dataset is truthfully represented each time the model is evaluated.

4.2.2 Metrics

We distinguish between metrics used for optimization versus final reporting.

- **Optimization (Threshold Independent):**

- **AUC-PR:** The primary objective for Optuna. It evaluates prediction ranking quality across all possible thresholds, ensuring robust class separation regardless of the final decision boundary.

- **Reporting (Threshold Dependent @ 0.5):**

- **F1-Score:** The primary production metric. Balances Precision and Recall to verify the model meets deployment standards.
- **Supplementary Metrics: G-Mean** (balancing sensitivity/specificity) and **Macro Recall** (class-averaged accuracy) are logged to ensure the minority class isn't ignored.

4.2.3 Hyperparameter Search Space

To find the optimal configuration, we defined the following search spaces for Optuna:

- **XGBoost:**

- **lambda & alpha (L2/L1 regularization):** $1e-8$ to 1.0 (log scale).
- **max_depth:** 1 to 9.
- **eta (Learning Rate):** $1e-8$ to 1.0 (log scale).
- **gamma (Min split loss):** $1e-8$ to 1.0 (log scale).
- **grow_policy:** depthwise or lossguide.
- **scale_pos_weight:** 1 to 100 (Critical for balancing the 1:12 class ratio).

- **TabNet:**

- **n_d & n_a (Latent widths):** 8 to 64.
- **n_steps (Decision steps):** 3 to 10.
- **gamma (Relaxation param):** 1.0 to 2.0.
- **lambda_sparse (Sparsity regularization):** $1e-6$ to 0.1 (log scale).
- **lr (Learning Rate):** 0.02 to 0.2.
- **mask_type:** entmax or sparsemax.
- **batch_size:** 256, 512, 1024, 2048.
- **virtual_batch_size:** 128, 256.

4.2.4 Experiments

We defined 3 different experiments to compare model performance under varying conditions:

1. **XGBoost**: Only trains XGBoost instances with the defined hyperparameter space.
2. **TabNet**: Only trains TabNet instances with the defined hyperparameter space.
3. **Combined**: Trains both models in parallel, allowing comparison of their best configurations.

Filter experiments by name		Q	Tag filter
☐	Name	Time created	Last modified
☐	Rice_Classification_Comparison	01/08/2026, 09:04:11 PM	01/08/2026, 09:04:11 PM
☐	Rice_Classification_TabNet	01/08/2026, 12:20:33 PM	01/08/2026, 12:20:33 PM
☐	Rice_Classification_XGBoost	01/08/2026, 11:38:51 AM	01/08/2026, 11:38:51 AM
☐	Default	01/03/2026, 02:19:51 PM	01/03/2026, 02:19:51 PM

Figure 7: Optuna Hyperparameter Optimization Experiments



Figure 8: Example Optuna Metrics Results for XGBoost Models

4.3 Model Results

The Optuna comparison experiment revealed that both architectures are highly capable of solving the task, with metrics exceeding 99% for both cases. However, **TabNet** was selected as the optimal model due to its marginally superior AUC-PR score, indicating better robustness in ranking probabilities. It was something to be expected from a deep learning model, but the good performance of a lightweight and efficient model like XGBoost is still noteworthy.

4.3.1 Performance Comparison

The best performing trials for each architecture were:

- **XGBoost:** Peak AUC-PR of 0.9998 and F1-Score of 0.9952.
- **TabNet:** Peak AUC-PR of 0.9999 and F1-Score of 0.9953.

4.3.2 Best Model Configuration

The final deployed TabNet model uses the following hyperparameters found by Optuna:

- **Architecture:** `n_d=15, n_steps=5, gamma=1.63`.
- **Regularization:** `lambda_sparse=0.0063, mask_type=entmax`.
- **Training:** `lr=0.198, batch_size=256`.

4.3.3 Operational Metrics

- **Validation Performance:**
 - **AUC-PR:** 0.9999
 - **F1-Score:** 0.9953
 - **G-Mean:** 0.9431 (Indicates strong balance between classes)
 - **Mean Recall:** 0.9455
- **Latency:** Sub-50ms inference time via ONNX Runtime.

4.4 Serialization

To ensure production compatibility and lightweight inference, we serialize the best model to **ONNX (Open Neural Network Exchange)** format.

- **Interception & Monkeypatching:** A critical challenge was newer versions of XGBoost exporting configuration parameters (like `base_score`) as JSON arrays (e.g., `"[0.5]"`), which caused `onnxmltools` to crash. We implemented a “Monkeypatch” on `xgb.Booster.save_config` to intercept the JSON string and normalize these values to scalars (e.g., `"0.5"`). It’s important to note that this patch will only work with the exact version of XGBoost used in the training environment and stored in the `pyproject.toml` file.
- **TabNet Wrapper:** For TabNet, we implemented a custom `torch.nn.Module` wrapper that standardizes the output to match the XGBoost template (returning both predicted labels and probability scores), ensuring the API code remains model-agnostic. This made the training process much simpler, as we just needed to add a flag to choose which model to train in the command line.

5 Automation Pipeline

The automation strategy relies on GitHub Actions to verify, build, and deploy the application.

5.1 Testing

Prior to any deployment, a `pytest` suite verifies system integrity:

1. **Data Tests:** Validates the data processing and plotting functions. It uses a mocker for the plots to simply ensure they are being called without errors.

2. **Model Tests:** Check ONNX loading, switching and predicting functionality. A mock model is used to ensure predictions are as expected.
3. **API Tests:** Similarly to the model tests, a mock model is used to ensure the API behaves close to the base function, and additionally checks for the metrics/ endpoint availability or some extra request related errors.

5.2 Continuous Integration (CI)

Run on every push and pull request to the main branch to ensure code quality before merging.

- **Linting & Formatting:** Enforces code style standards using black and then pylint(via Makefile).
- **Automated Testing:** Executes the pytest suite described above.

5.3 Continuous Deployment (CD)

We maintain two distinct deployment pipelines handled by separate workflows.

5.3.1 Backend Pipeline (API)

This pipeline deploys the inference endpoints to **Render** and implements a “Conditional Training” strategy logic: The workflow inspects the commit message for the #TRAIN tag.

- **Training Mode:** If detected, the full pipeline runs: Optuna optimization, model serialization (ONNX), and Docker image rebuilding. Because TabNet training is very time-consuming, we only train XGBoost models in this mode to keep the pipeline efficient. To still be able to use both models through the API, a local serialization of the best TabNet model is performed and added to the Docker image without retraining.
- **Standard Mode:** If absent, training is skipped, and the existing code is packaged.

The Docker image is then pushed to **Docker Hub** and a webhook triggers **Render** to pull the new image and update the live service.

5.3.2 Frontend Pipeline (UI)

The user interface is deployed to **Hugging Face Spaces** from the ui-frontend branch. Some notable features of this pipeline include:

- **Large File Support:** Explicitly handles git lfs for static assets, which in our case is an .mp3 file used in the visualization component of the UI.
- **Orphan Deployment:** To ensure a clean state on the Spaces remote, the CI creates a temporary orphan branch and force-pushes the application, synchronizing the space with the latest repository version. This prevents possible branch history conflicts or residual files from previous commits.

6 Monitoring

We didn’t mention monitoring in the previous section, as the deployment of prometheus and grafana is done manually, given that these services are not expected to change often, and automating their deployment would add unnecessary complexity to the CD pipeline.

6.1 Prometheus

It's a simple service that scrapes the `/metrics` endpoint of the API every 15 seconds. Not much customization was needed here, just the basic configuration to point to the API service. It has its own Docker container, and is deployed to **Render** a separate render web service.

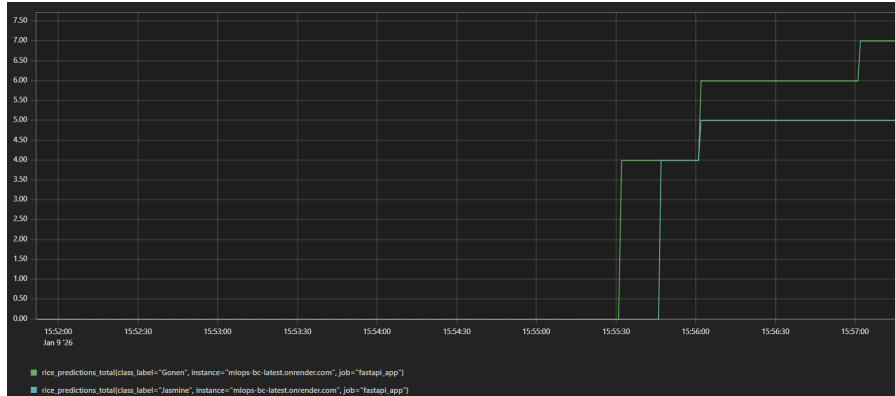


Figure 9: Example of prometheus scraping counters from the API

The counters exposed by the API include:

- `rice_predictions_total{variety="gonen"}`: Total number of **Gonen** predictions made.
- `rice_predictions_total{variety="jasmine"}`: Total number of **Jasmine** predictions made.
- `active_model_info`: Labels indicating the currently active model versions.
- `model_f1_score`: The static F1-Score of the currently deployed model on the validation set.
- `model_auc_pr`: The static AUC-PR of the currently deployed model on the validation set.
- `model_mean_recall`: The static Macro Recall of the currently deployed model on the validation set.

6.2 Grafana

Here one can visualize these metrics in a real-time through a preconfigured dashboard, which is defined in a JSON file stored in the `graf/` directory of the repository. It is also deployed to **Render** as a separate web service, in its own Docker container.

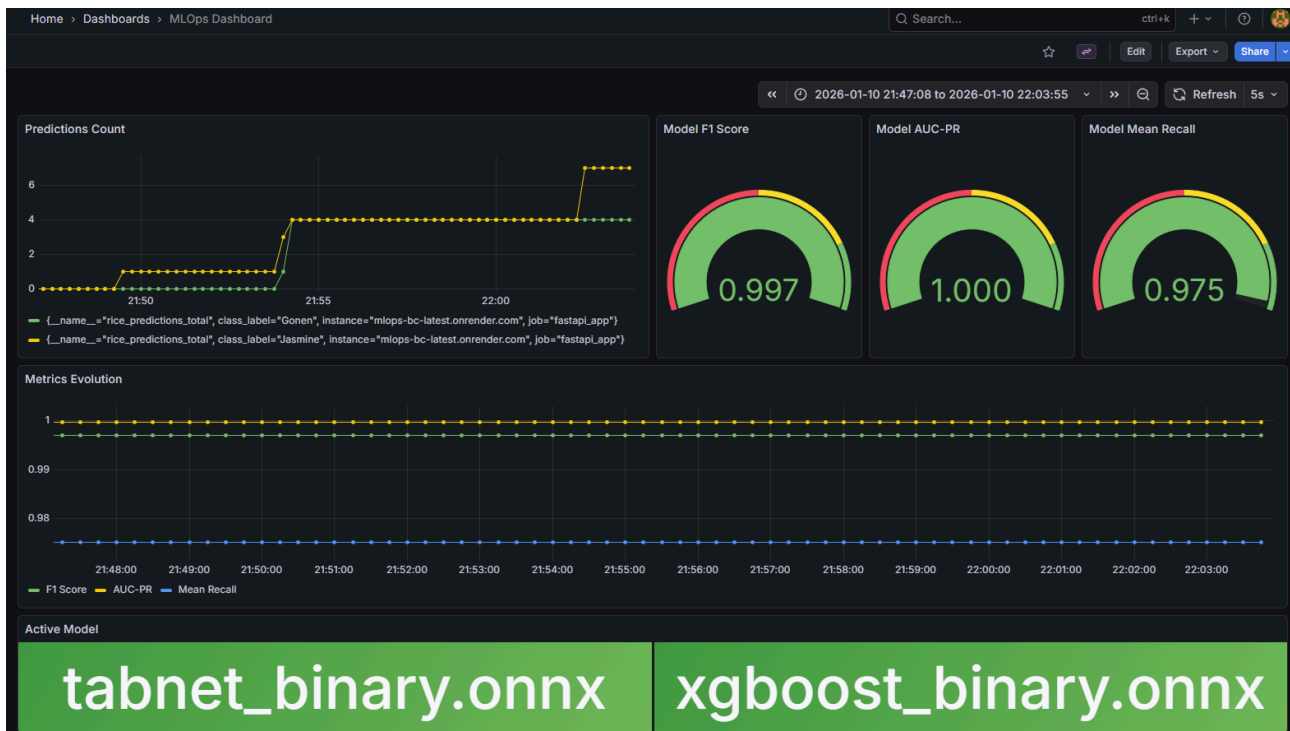


Figure 10: Grafana Dashboard Visualizing Model Metrics

Key indicators include:

- **Prediction Distribution:** Real-time count of Gonen vs. Jasmine predictions which updates as the API receives new requests (rice_predictions_total).
- **Model Quality:** The static validation F1-Score of the running model (model_f1_score), ensuring operations teams know the expected performance baseline.
- **Active Model:** Tracks exactly which model version is serving requests.

Upon access, users are greeted with a login screen. The default credentials are admin/admin.

It's important to note that all the services (API, Prometheus, Grafana) are deployed on free-tier Render web services, which lead to data loss after periods of inactivity. Therefore, the monitoring dashboards will only display data collected after the last service restart.

7 Personal Server

In addition to the cloud-based deployments, we have also deployed the complete stack on a local **Ubuntu Server** running in our home infrastructure. This provides a persistent, fully-controlled environment for testing and demonstration purposes.

7.1 Infrastructure Setup

The local deployment utilizes the following components:

- **Ubuntu Server:** Base operating system hosting all containerized services.
- **Portainer:** Web-based container management platform used to deploy and manage the Docker Compose stack.
- **Dozzle:** Lightweight log viewer for real-time container monitoring.

- **Cloudflare Tunnel:** Secure tunnel solution to expose local services to the internet without port forwarding, connected to our existing domain.

7.2 Docker Compose Stack

We created a unified `docker-compose.yml` file that orchestrates both the API and frontend services:

- **API Service:** Uses the same Docker image (`yngvin/mlops-bc:latest`) deployed on Render, running the FastAPI inference endpoint on port 8000.
- **Frontend Service:** A new Node.js-based Docker image (`yngvin/mlops-rice-ui:latest`) hosting the interactive rice grain visualization interface, exposed on port 5003.

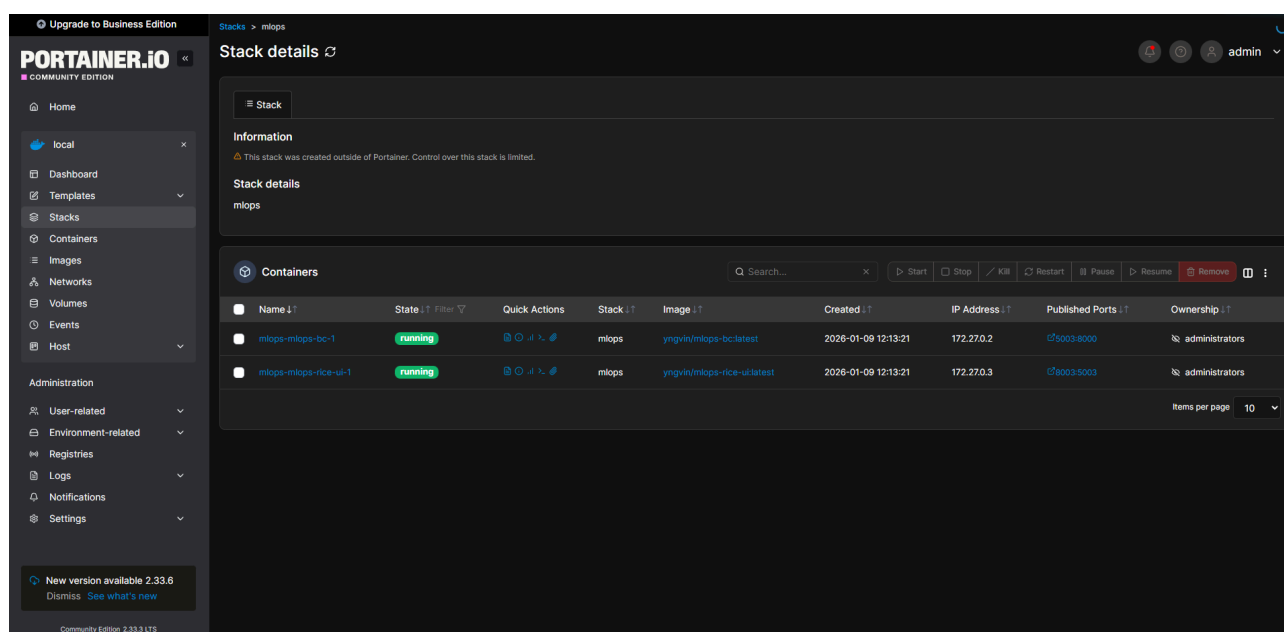


Figure 11: Portainer Dashboard showing the mlops stack with running containers

The stack is managed entirely through Portainer’s web interface, which provides a simple point-and-click deployment experience. Both containers run successfully as shown in the dashboard, with health status and resource usage visible at a glance.

7.3 Container Monitoring

For operational visibility, we use **Dozzle** to monitor container logs in real-time without needing SSH access:

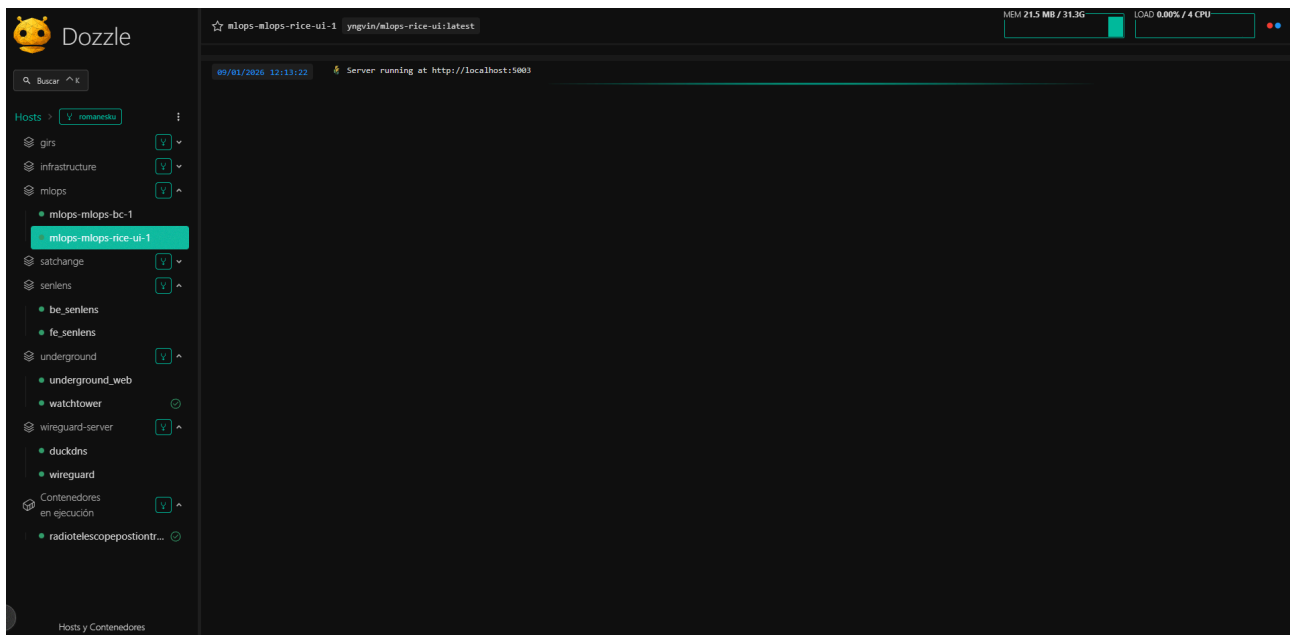


Figure 12: Dozzle monitoring the rice-ui container running on port 5003

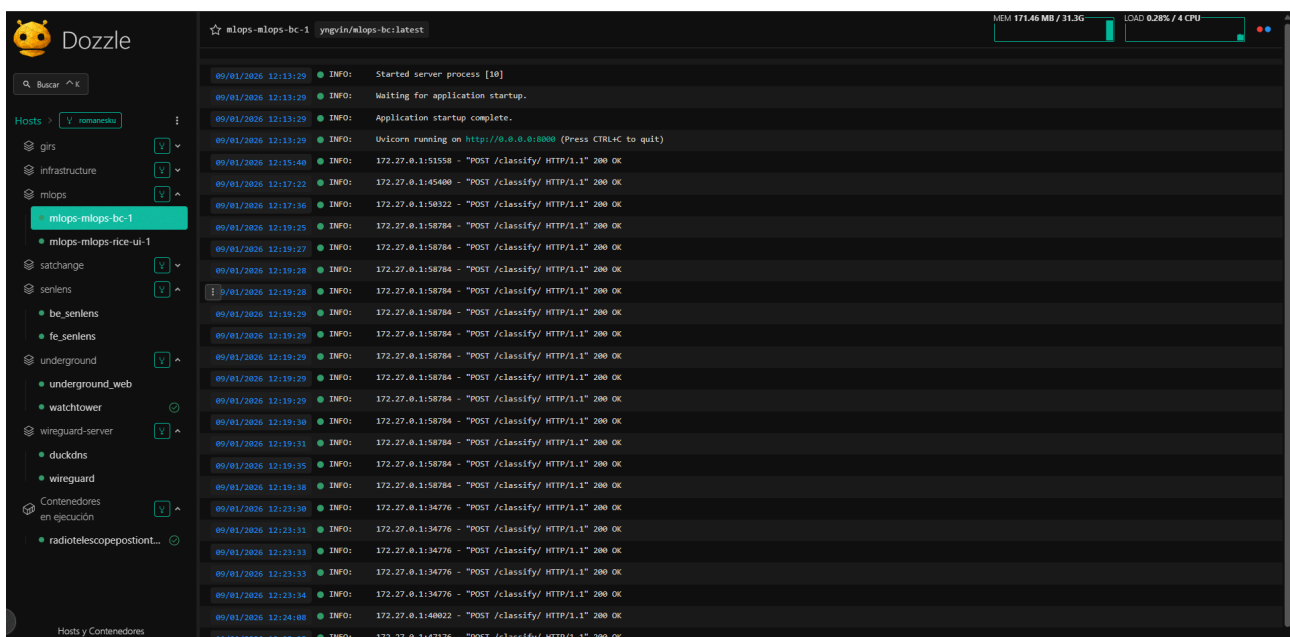


Figure 13: Dozzle monitoring the API container with Uvicorn logs on port 8000

These logs confirm both services are operational and accepting connections.

7.4 Public Access via Cloudflare Tunnel

To make the local deployment accessible over the internet, we configured a **Cloudflare Tunnel** that creates secure, encrypted connections from our Ubuntu Server to Cloudflare's edge network. This eliminates the need for port forwarding or exposing our home IP address.

Two published application routes were configured:

- `rice.endikens.com`: Points to `http://localhost:5003` (Frontend UI)
- `apirice.endikens.com`: Points to `http://localhost:8003` (API endpoint)

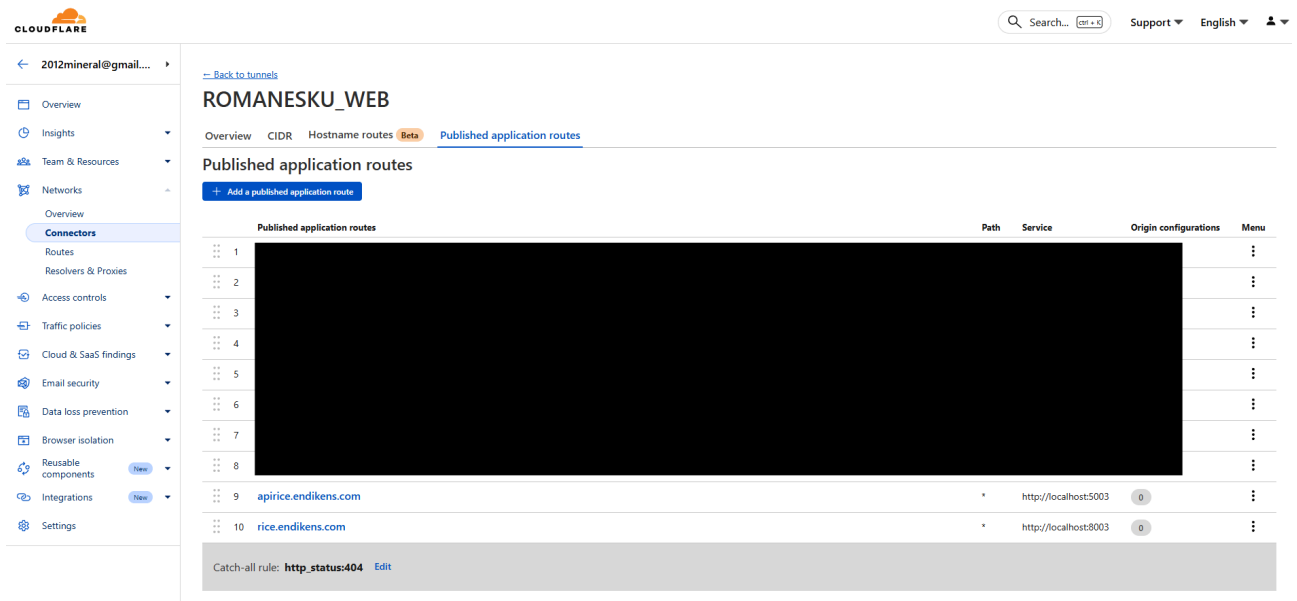


Figure 14: Cloudflare Tunnel configuration with published application routes

This setup provides production-like accessibility while maintaining full control over the infrastructure.

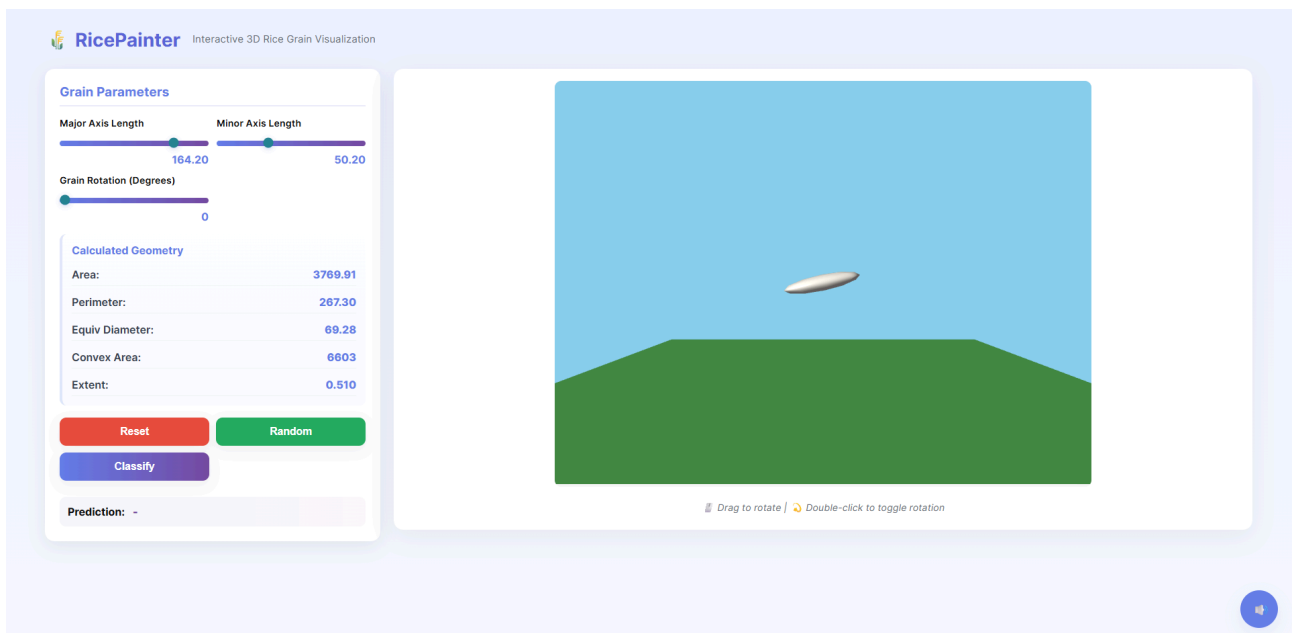


Figure 15: Rice Painter interactive visualization running on the personal server

8 Repository Links

- **GitHub Repository:** <https://github.com/Yngvine/MLOPs-Binary-Classicator>
- **Docker Hub (API):** <https://hub.docker.com/repository/docker/yngvin/mlops-bc>
- **Docker Hub (Grafana):** <https://hub.docker.com/repository/docker/yngvin/mlops-bcg>
- **Docker Hub (Prometheus):** <https://hub.docker.com/repository/docker/yngvin/mlops-bcp>
- **Docker Hub (Server deployment):** <https://hub.docker.com/repository/docker/yngvin/mlops-rice-ui>

- **Render API Service:** <https://mlops-bc-latest.onrender.com>
- **Render Grafana Service:** <https://mlops-bcg-latest.onrender.com>
- **Render Prometheus Service:** <https://mlops-bcp.onrender.com>
- **Hugging Face Space (UI):** <https://huggingface.co/spaces/Yngvine/MLOPs-BC>
- **Private Server Deployment:** <https://rice.endikens.com/>